ENUMERATIONS IN C

Days of the Week and Traffic Light Programs

Name	**:**	Rudra Pratap Singh
SAP ID	**:**	590032811
Batch	**:**	16

Question 1: Enumeration for Days of the Week

Problem Statement

Create an enumeration for the days of the week (SUNDAY to SATURDAY) and print each day along with its integer value.

Input: No Input

Expected Output: SUNDAY = 0, MONDAY = 1, TUESDAY = 2, WEDNESDAY = 3, THURSDAY = 4, FRIDAY = 5, SATURDAY = 6

Theory

An enumeration (enum) in C is a user-defined data type that consists of a set of named integer constants. By default, the first name in an enum is given the value 0, and every following name gets a value one greater than the previous one. This makes programs easier to read because meaningful names are used instead of plain numbers.

Algorithm

1. Start the program.
2. Declare an enum Day with the constants SUNDAY, MONDAY, TUESDAY, WEDNESDAY, THURSDAY, FRIDAY and SATURDAY.
3. Create an array of strings that stores the name of each day.
4. Run a loop from SUNDAY (0) to SATURDAY (6).
5. In each iteration, print the day name and its integer value.
6. Print a comma and a space after every day except the last one.
7. Stop the program.

Program

```c
#include <stdio.h>

/* Enumeration for the days of the week (values start from 0) */
enum Day {
    SUNDAY,      /* 0 */
    MONDAY,      /* 1 */
    TUESDAY,     /* 2 */
    WEDNESDAY,   /* 3 */
    THURSDAY,    /* 4 */
    FRIDAY,      /* 5 */
    SATURDAY     /* 6 */
};

int main(void)
{
    /* Array of names so that each enum value can be printed as text */
    const char *names[] = {"SUNDAY", "MONDAY", "TUESDAY", "WEDNESDAY",
                           "THURSDAY", "FRIDAY", "SATURDAY"};
    int d;

    for (d = SUNDAY; d <= SATURDAY; d++)
    {
        printf("%s = %d", names[d], d);
        if (d != SATURDAY)
            printf(", ");
    }
    printf("\n");

    return 0;
}
```

Output

```
$ gcc q1.c -o q1
$ ./q1
SUNDAY = 0, MONDAY = 1, TUESDAY = 2, WEDNESDAY = 3, THURSDAY = 4, FRIDAY = 5, SATURDAY = 6
```

Explanation

- enum Day { SUNDAY, ... SATURDAY }; automatically assigns SUNDAY = 0, MONDAY = 1, and so on up to SATURDAY = 6.
- Because enum constants are integers, they can be used directly as the loop counter (d) and as the index of the names[] array.
- printf("%s = %d", names[d], d) prints the day name followed by its integer value.
- The if condition (d != SATURDAY) prevents an extra comma after the last day.

Question 2: Enumeration for Traffic Lights

Problem Statement

Define an enumeration for traffic lights (RED, YELLOW, GREEN) and print 'Stop', 'Wait' or 'Go' based on its value.

Input: GREEN

Expected Output: Go

Theory

An enum can be combined with a switch statement to make decisions based on named constants. In this program, the three traffic light colours are stored as an enum, and the action to take (Stop, Wait or Go) is chosen by checking the enum value. Since C cannot read an enum name directly from the keyboard, the colour is read as a string and then converted into the matching enum value.

Algorithm

1. Start the program.
2. Declare an enum TrafficLight with the constants RED, YELLOW and GREEN.
3. Read the colour entered by the user as a string.
4. Compare the string with "RED", "YELLOW" and "GREEN" and store the matching enum value in a variable.
5. If the colour does not match any of them, print an error message and stop.
6. Use a switch statement: print "Stop" for RED, "Wait" for YELLOW and "Go" for GREEN.
7. Stop the program.

Program

```
#include <stdio.h>
#include <string.h>

/* Enumeration for traffic light colours */
enum TrafficLight {
    RED,      /* 0 */
    YELLOW,   /* 1 */
    GREEN     /* 2 */
};

int main(void)
{
    char input[20];
    enum TrafficLight light;

    printf("Enter traffic light colour (RED / YELLOW / GREEN): ");
    scanf("%19s", input);

    /* Convert the entered text into the matching enum value */
    if (strcmp(input, "RED") == 0)
        light = RED;
    else if (strcmp(input, "YELLOW") == 0)
        light = YELLOW;
    else if (strcmp(input, "GREEN") == 0)
        light = GREEN;
    else
    {
        printf("Invalid colour entered.\n");
        return 1;
    }

    /* Decide the action based on the enum value */
    switch (light)
    {
        case RED:
            printf("Stop\n");
            break;
        case YELLOW:
            printf("Wait\n");
            break;
        case GREEN:
            printf("Go\n");
            break;
    }

    return 0;
}
```

Output

Sample Run 1 (Given Input):

```
$ gcc q2.c -o q2
$ ./q2
Enter traffic light colour (RED / YELLOW / GREEN): GREEN
Go
```

Sample Run 2:

```
Enter traffic light colour (RED / YELLOW / GREEN): RED
Stop
```

Sample Run 3:

```
Enter traffic light colour (RED / YELLOW / GREEN): YELLOW
Wait
```

Sample Run 4 (Invalid Input):

```
Enter traffic light colour (RED / YELLOW / GREEN): PURPLE
Invalid colour entered.
```

Explanation

- enum TrafficLight { RED, YELLOW, GREEN }; gives RED = 0, YELLOW = 1 and GREEN = 2.
- scanf("%19s", input) reads the colour as text, and the limit of 19 characters prevents buffer overflow.
- strcmp() compares the entered text with each colour name and returns 0 when both are equal.
- The switch statement selects the correct message based on the enum value: RED prints Stop, YELLOW prints Wait and GREEN prints Go.
- Input other than RED, YELLOW or GREEN is handled by printing an error message.

Conclusion

In this assignment, enumerations were used to give meaningful names to integer constants. The first program showed how enum values start from 0 and increase by 1, and how they can be used in loops. The second program showed how enums work together with the switch statement to make decisions. Both programs were compiled and tested, and they produce the expected output.